Q 1)

Data Structures Project: English Dictionary Using an Array of Linked Lists

Course: Data Structures
Programming Language: C++
Level: Intermediate
Data Structures: Array + Singly Linked Lists
Project Title: Alphabetical Dictionary Management System

1. Project Description

Develop a C++ application that stores and manages English words using a combination of an **array and singly linked lists**.

The application must use a fixed-size array containing **26 elements**, representing the English alphabet from A to Z.

Each array element contains a pointer to the head of a singly linked list that stores words beginning with the corresponding letter.

For example:

- Array index 0 stores words beginning with A.
- Array index 1 stores words beginning with B.
- Array index 2 stores words beginning with C.
- ...
- Array index 25 stores words beginning with Z.

The linked lists must be created dynamically using pointers.

2. Data Structure Design

Students must use the following structure:

```
struct Node {
   string word;
   Node* next;
};
```

The main dictionary contains an array of 26 pointers:

Node* dictionary[26];

Initially, all pointers must be nullptr.

Example of the Data Structure

ARRAY OF 26 POINTERS

==========

Index	Letter	Linked List
0	A	---> Apple → Ant → Air → NULL
1	B	---> Banana → Book → Ball → NULL
2	C	---> Cat → Car → Computer → NULL
3	D	---> Dog → Door → NULL
4	E	---> Elephant → Egg → NULL
5	F	---> Fish → Flower → NULL
.	.	.
.	.	.
.	.	.

24 Y ---> Yellow → Yes → NULL

25 Z ---> Zebra → Zoo → NULL

Important: Each array element represents a separate singly linked list.

3. Project Requirements

Students must implement the following operations.

No.	Function	Description
1	addWord()	Insert a word into the appropriate linked list
2	deleteWord()	Delete a specific word
3	searchWord()	Search for a word in the dictionary
4	displayLetterWords()	Display all words beginning with a specific letter
5	displayAllWords()	Display all dictionary words from A to Z
6	countWordsByLetter()	Count words beginning with a specific letter
7	countAllWords()	Count the total number of words
8	findLongestWord()	Find the longest word in the dictionary

Q 2)

Printer Spooler and Print Job Recovery System

Data Structures: Queue + Stack (Singly Linked Lists)

Real-World Scenario

A company has several employees sharing one network printer.

Employees send print jobs to the printer throughout the day.

The printer processes jobs in the order they arrive.

Occasionally, a print job fails because of:

- Paper shortage
- Printer disconnection
- Printing errors

The system must maintain a queue of pending jobs and a stack of failed jobs that can be retried.

Project Requirements

Implement:

Print Queue: Stores jobs waiting to be printed.

Failed Jobs Stack: Stores failed jobs for recovery.

Print Job Information

```
struct PrintJob {
   int jobID;
   string employeeName;
   string documentName;
   int numberOfPages;
   int submissionTime;
};
```

Required Operations

1. submitPrintJob() – Add a new print job.
2. processPrintJob() – Print the first job.
3. displayPendingJobs() – Display the print queue.
4. cancelPrintJob() – Cancel a pending job.
5. simulatePrinterFailure() – Simulate a printing error.
6. retryFailedJob() – Retry the most recently failed job.
7. displayFailedJobs() – Display failed jobs.
8. calculateTotalPages() – Calculate the number of pages waiting to be printed.

9. displayStatistics() – Display completed, failed, and pending jobs.

Example

========== PRINTER MANAGEMENT ==========

1. Submit Print Job

2. Process Print Job

3. Pending Jobs

4. Cancel Job

5. Simulate Failure

6. Retry Failed Job

7. Failed Jobs

8. Statistics

9. Exit

You can upgrade the **Printer Spooler and Print Job Recovery System** by replacing the normal FIFO Queue with a **Priority Queue implemented using a Singly Linked List**.

FRONT

|

v

[102, P1] → [104, P1] → [103, P2] → [101, P3] → [105, P3] → NULL

|

Highest Priority

Printer Spooler – Shortest Job First (SJF)

You can modify the **Priority Printer Management System** to use **Shortest Job First (SJF)** scheduling.

Instead of assigning priorities manually, the printer processes jobs according to their **number of pages**.

Scheduling Rules

1. The job with the **fewest pages** is printed first.
2. If two jobs have the same number of pages, the job submitted first is printed first (**FIFO**).
3. Failed jobs are moved to a **Stack** implemented using a Singly Linked List.
4. Retried jobs return to the Priority Queue according to their number of pages.
5. A job already being printed is not interrupted by a newly submitted shorter job (non-preemptive SJF).

Priority Queue after insertion:

```
</> Plain text

FRONT
  |
  v
[105, 2] → [102, 5] → [104, 5] → [103, 12] → [101, 20] → NULL
```

Q 3)

1. Project Description

Develop a C++ application that evaluates mathematical expressions entered by the user.

The application must use **Stacks implemented manually using Singly Linked Lists**.

The program should support arithmetic expressions containing multiple operators, parentheses, and numbers.

Example

Input:

(10 + 5) * 2 - 8 / 4

Output:

Result = 28

2. Supported Operators

Operator	Operation	Priority
+	Addition	1
-	Subtraction	1
*	Multiplication	2
/	Division	2
^	Exponentiation	3
( )	Parentheses	Highest grouping priority

The program must respect operator precedence and associativity. Exponentiation is right-associative.

3. Project Requirements

Students must implement the following features:

1. Read a mathematical expression from the user.
2. Validate the expression.
3. Check whether parentheses are balanced.
4. Convert an infix expression into postfix notation.
5. Evaluate the postfix expression using a Stack.
6. Display the final result.
7. Handle division by zero.
8. Support multi-digit integers and decimal numbers.
9. Handle spaces between numbers and operators.
10. Allow users to evaluate multiple expressions.

For this version, negative numbers and unary operators are optional advanced features.

4. Required Data Structures

Students must implement a generic Stack class using a Singly Linked List.

```
template <class T>
struct Node {
   T data;
   Node<T>* next;
};
```

Stack Class

```
template <class T>
class Stack {
private:
   Node<T>* top;

public:
   Stack();
   ~Stack();

   void push(T value);
   T pop();
   T peek();

   bool isEmpty();
   void display();
};
```

Students are not allowed to use the STL stack or list containers.

5. Required Algorithms

Algorithm 1: Parentheses Validation

Input:

((10 + 5) * (8 - 3))

Output:

Parentheses are balanced.

Input:

((10 + 5) * (8 - 3)

Output:

Error: Unbalanced parentheses.

Algorithm 2: Infix to Postfix Conversion

Students must implement an infix-to-postfix conversion algorithm using an operator Stack.

Example:

Infix:

(10 + 5) * 2 - 8 / 4

Postfix:

10 5 + 2 * 8 4 / -

Algorithm 3: Postfix Evaluation

Students must evaluate postfix expressions using an operand Stack.

Example:

Postfix: 10 5 + 2 * 8 4 / -

Evaluation:

Step Token Stack (Bottom → Top)

1 10 10

2 5 10, 5

Step Token Stack (Bottom → Top)

3	+	15
4	2	15, 2
5	*	30
6	8	30, 8
7	4	30, 8, 4
8	/	30, 2
9	-	28

Final Result: 28

6. Required Functions

Function	Description
isBalanced()	Check balanced parentheses
precedence()	Return operator precedence
isOperator()	Check whether a character is an operator
infixToPostfix()	Convert infix to postfix
evaluatePostfix()	Evaluate a postfix expression
applyOperator()	Perform an arithmetic operation
validateExpression()	Validate expression syntax
displaySteps()	Display evaluation steps

7. Application Menu

```
====================================
   ARITHMETIC EXPRESSION EVALUATOR
====================================
```

1. Enter Mathematical Expression

2. Check Balanced Parentheses

3. Convert Infix to Postfix

4. Evaluate Expression

5. Display Evaluation Steps

6. Exit

Enter your choice:

8. Test Cases

Expression	**Expected Result**
10 + 5 * 2	20
(10 + 5) * 2	30
100 / (5 * 2)	10
2 ^ 3 ^ 2	512
20 - 5 - 3	12
15 / 2	7.5
(8 + 2) * (6 - 3)	30
10 / 0	Error
(5 + 3	Error
10 + * 5	Error